



Universidad de Santiago de Chile  
Facultad de Ciencias  
Departamento de Física  
Astrofísica con mención en Ciencia de Datos

# Proyecto Sonda Parker Solar Probe

## Informe Final

*Martina Molina Rizzo, Thiare Molina Tapia  
Bárbara Quilodrán Hijerra, Ivannia Villalba Berrios*

**Asignatura:** Desarrollo de Software — **Profesor:** Ricardo Hasbun  
24 de Agosto de 2026

# Índice

|                                                                     |           |
|---------------------------------------------------------------------|-----------|
| <b>1. Introducción</b>                                              | <b>3</b>  |
| <b>2. Identificación del cliente</b>                                | <b>3</b>  |
| 2.1. Perfil y Rol en el Proyecto . . . . .                          | 3         |
| 2.2. Mecanismos de Interacción y Feedback . . . . .                 | 3         |
| 2.3. Criterios de Aceptación del Cliente . . . . .                  | 3         |
| <b>3. Descripción y Justificación del Problema</b>                  | <b>4</b>  |
| 3.1. Descripción de la Necesidad . . . . .                          | 4         |
| 3.2. Justificación de la Solución Desarrollada . . . . .            | 4         |
| <b>4. Objetivos del proyecto</b>                                    | <b>4</b>  |
| 4.1. Objetivo General . . . . .                                     | 4         |
| 4.2. Objetivos Específicos . . . . .                                | 4         |
| <b>5. Evaluación de Requerimientos del Sistema</b>                  | <b>5</b>  |
| 5.1. Requerimientos Funcionales . . . . .                           | 5         |
| 5.2. Requerimientos No Funcionales . . . . .                        | 5         |
| <b>6. Alcance de la Solución</b>                                    | <b>5</b>  |
| 6.1. Producto Mínimo Viable (MVP) . . . . .                         | 5         |
| 6.1.1. Funcionalidades Incluidas en el MVP . . . . .                | 6         |
| 6.2. Funcionalidades Excluidas . . . . .                            | 6         |
| 6.3. Justificación del Alcance . . . . .                            | 6         |
| <b>7. Metodología de trabajo</b>                                    | <b>6</b>  |
| 7.1. Estructura del proceso . . . . .                               | 7         |
| 7.2. Herramientas y entorno de desarrollo . . . . .                 | 7         |
| 7.3. Burndown Chart . . . . .                                       | 7         |
| 7.4. Gestión del Backlog y Cierre de Historias de Usuario . . . . . | 9         |
| 7.4.1. Product Backlog (Estado Final) . . . . .                     | 9         |
| 7.4.2. Sprint Backlog (Sprint 3) . . . . .                          | 9         |
| <b>8. Diseño de la Arquitectura</b>                                 | <b>9</b>  |
| 8.1. Atributos de Calidad y Rendimiento Realizados . . . . .        | 10        |
| 8.2. Código fuente completo y ejecutable en GitHub . . . . .        | 10        |
| <b>9. Flujo de Ejecución (Implementación)</b>                       | <b>11</b> |
| <b>10.Pruebas unitarias y/o de integración</b>                      | <b>11</b> |
| 10.1. Pipeline de integración continua . . . . .                    | 11        |
| <b>11.Pruebas pytest</b>                                            | <b>12</b> |
| <b>12.Archivo pyproject.toml</b>                                    | <b>14</b> |

|                                                               |           |
|---------------------------------------------------------------|-----------|
| <b>13.Manual Básico de Uso</b>                                | <b>14</b> |
| 13.1. Requisitos Previos del Sistema . . . . .                | 14        |
| 13.2. Instalación de Dependencias . . . . .                   | 14        |
| 13.3. Flujo de Ejecución y Uso . . . . .                      | 14        |
| <b>14.Limitaciones de la solución</b>                         | <b>16</b> |
| <b>15.Conclusiones del proyecto</b>                           | <b>17</b> |
| <b>16.Anexos</b>                                              | <b>17</b> |
| 16.1. Declaración de uso de Inteligencia Artificial . . . . . | 17        |

# 1. Introducción

Este documento presenta los resultados finales del proyecto, culminando en el Sprint 3. Tras consolidar en las etapas anteriores nuestro flujo de trabajo en GitHub y el procesamiento de datos extraídos desde los repositorios de la NASA (SPDF/CDAWeb) [1], esta fase de cierre se enfocó en integrar y desplegar el software definitivo para nuestra cliente, la investigadora Matilde Coello [4].

En este informe detallamos cómo acoplamos el *backend* en Python con la nueva interfaz gráfica (`index.html`). Además, explicamos la implementación final de los filtros temporales, la conversión dinámica de unidades astronómicas y la exportación de resultados a formato `.txt`. Finalmente, documentamos las pruebas automatizadas y de rendimiento que garantizan la estabilidad de la herramienta, asegurando que el sistema no solo cumpla con los 7 segundos máximos de respuesta, sino que elimine por completo el tedioso proceso manual que afectaba la investigación de nuestra cliente.

## 2. Identificación del cliente

Nuestra cliente principal es **Matilde Coello** [4], investigadora en el área de la heliofísica centrada en el análisis de las mediciones de los perihelios de la Sonda Parker Solar Probe (PSP), quien además se desempeña como académica en la Universidad de Santiago de Chile [4].

### 2.1. Perfil y Rol en el Proyecto

En el marco del desarrollo ágil de este software, la investigadora asumió el rol de *Product Owner* y usuaria final. Su participación fue clave para establecer las prioridades del *backlog*, definir las historias de usuario y validar los criterios de aceptación en cada ciclo de desarrollo.

### 2.2. Mecanismos de Interacción y Feedback

Durante el ciclo de vida del proyecto (Sprints 1, 2 y 3), la comunicación con la cliente se estructuró mediante:

- **Reuniones de avance (Sprints Demos):** Presentación de prototipos funcionales al cierre de cada sprint para recibir retroalimentación directa sobre la interfaz y las funcionalidades.
- **Validación de requerimientos:** Definición conjunta de las unidades de distancia requeridas (radios solares, unidades astronómicas y radios terrestres) y los parámetros de salida tabulares.

### 2.3. Criterios de Aceptación del Cliente

Para dar por concluido exitosamente el proyecto en este Sprint 3, la investigadora definió un conjunto de criterios clave que la solución debía cumplir:

1. **Automatización y filtrado:** Seleccionar rangos de fechas específicos a través de una interfaz gráfica sin manipular archivos crudos manualmente.

2. **Exportación directa:** Obtener los datos consolidados en un archivo limpio con extensión `.txt`.
3. **Eficiencia de tiempo:** Reducir el tiempo de obtención y visualización de datos a un máximo de 7 segundos por consulta.

## 3. Descripción y Justificación del Problema

### 3.1. Descripción de la Necesidad

Los datos de los perihelios de la sonda Parker Solar Probe y sus mediciones de distancia se encuentran distribuidos públicamente en la web de la NASA (SPDF/CDAWeb), pero carecen de un formato de fácil descarga y manipulación directa para los investigadores [5]. Nuestra cliente, la investigadora Matilde Coello, requería extraer la información de la tabla *'PSP perihelia'* —donde la columna *'Orbit'* actúa como marcador temporal natural— para consolidarla en un formato manejable (`.txt`). Adicionalmente, el trabajo de investigación exigía flexibilidad para visualizar y exportar las distancias en tres unidades de medida específicas: radios solares, unidades astronómicas y radios terrestres.

### 3.2. Justificación de la Solución Desarrollada

Para resolver esta problemática, se construyó e implementó una aplicación web que centraliza y automatiza la consulta de dichas fuentes científicas, entregando la información procesada en un único punto de acceso. Esta solución redujo drásticamente el tiempo de obtención de datos y aseguró la reproducibilidad de los resultados, garantizando que el equipo de investigación trabaje con información estandarizada bajo los mismos parámetros. La herramienta desarrollada no busca reemplazar las fuentes oficiales de la NASA, sino optimizar el flujo de trabajo, habiendo transformado exitosamente un proceso manual y fragmentado en uno completamente automatizado y confiable.

## 4. Objetivos del proyecto

### 4.1. Objetivo General

Diseñar, implementar y validar una aplicación web basada en Python que permite consultar, visualizar y descargar datos de la posición de la sonda Parker Solar Probe (PSP) respecto al Sol durante intervalos de tiempo específicos.

### 4.2. Objetivos Específicos

- Extraer y automatizar la obtención de datos crudos desde los repositorios científicos oficiales de la NASA (SPDF/CDAWeb) [5].
- Procesar y estructurar la información temporal (*epoch*) y de posición (RTN) mediante *DataFrames* de la librería Pandas, asegurando la limpieza e interpolación de vacíos de datos.
- Implementar un conversor dinámico de unidades de distancia que permite alternar entre radios solares, unidades astronómicas y radios terrestres.

- Desplegar una interfaz web funcional (`index.html`) que facilite la selección de rangos de fechas y presente los datos procesados en formato tabular.
- Proveer un mecanismo de exportación directa que permite descargar la información filtrada y consolidada en formato `.txt`.

## 5. Evaluación de Requerimientos del Sistema

### 5.1. Requerimientos Funcionales

1. **RF1 - Selección de rango temporal:** El sistema permite al usuario seleccionar una fecha o un intervalo de fechas específico mediante el filtro de la interfaz web (`index.html`). (*Cumplido*)
2. **RF2 - Visualización tabular:** El sistema genera e imprime en pantalla una tabla con las columnas de tiempo (*epoch*) y posición de la sonda PSP respecto al Sol (RTN), además de velocidad, densidad y temperatura. (*Cumplido*)
3. **RF3 - Exportación de datos:** El sistema incluye la funcionalidad completa para descargar la tabla filtrada en formato `.txt`. (*Cumplido*)

### 5.2. Requerimientos No Funcionales

1. **RNF1 - Lenguaje de desarrollo:** El proyecto fue desarrollado en su totalidad utilizando Python (versión 3.9+) en el *backend*. (*Cumplido*)
2. **RNF2 - Procesamiento eficiente:** Los datos crudos en formato `.cdf` son procesados y estructurados de forma eficiente utilizando *DataFrames* de la librería Pandas. (*Cumplido*)
3. **RNF3 - Trazabilidad y credibilidad:** Se indica explícitamente en la interfaz la fuente oficial de extracción (repositorio CDAWeb / SPDF de la NASA [1]). (*Cumplido*)
4. **RNF4 - Tiempo de respuesta:** Mediante la optimización del filtrado local y caché de datos, el tiempo de respuesta de la consulta se mantiene por debajo del límite exigido de 7 segundos. (*Cumplido*)

## 6. Alcance de la Solución

### 6.1. Producto Mínimo Viable (MVP)

El producto mínimo viable de este proyecto consiste en una herramienta que automatiza la obtención de datos científicos de la Sonda Parker Solar Probe, permitiendo a la investigadora consultar, visualizar y exportar información de los perihelios sin depender de la navegación manual por los repositorios de la NASA (SPDF/CDAWeb) [1], optimizando significativamente el tiempo de investigación.

### 6.1.1. Funcionalidades Incluidas en el MVP

El MVP integra el flujo completo de datos desde el *backend* hasta el *frontend*:

- **Conexión y descarga automatizada** (`descargar_data.py`): Conecta con NASA SPDF/CDAWeb y descarga los archivos `.cdf` de los perihelios, incluyendo una verificación previa del espacio en disco disponible.
- **Lectura y limpieza de datos** (`Filtrar_data.py`): Estructura los datos crudos en *DataFrames* de Pandas, elimina lecturas físicas inválidas, calcula la distancia al Sol, detecta perihelios y remuestrea la serie temporal a intervalos de 1 hora mediante interpolación lineal para cubrir vacíos de transmisión.
- **Interfaz web** (`index.html`): Permite seleccionar un intervalo de fechas, visualizar los datos en formato tabular y convertir las distancias entre radios solares, unidades astronómicas y radios terrestres.
- **Exportación de datos**: Dispone de un botón funcional para la descarga de la tabla generada en formato `.txt`.

Este flujo integral permite dar cumplimiento a las cuatro historias de usuario iniciales (HU1–HU4) y satisface los requerimientos funcionales definidos al inicio del proyecto.

### 6.2. Funcionalidades Excluidas

Con el fin de acotar el desarrollo al tiempo disponible, se excluyeron explícitamente del alcance las siguientes características:

- Modelos predictivos de las órbitas.
- Visualización de las posiciones relativas de Mercurio, Venus y la Tierra.

### 6.3. Justificación del Alcance

El subconjunto de funcionalidades implementadas constituye la base crítica de la herramienta. Sin alguno de los componentes descritos, el sistema no cumpliría su objetivo central: automatizar el proceso de recolección de datos. Por otro lado, las funcionalidades excluidas no resultaban indispensables para el flujo de trabajo primario de la cliente (**Matilde Coello** [4]), quien tras evaluar las demostraciones dio una retroalimentación positiva destacando la utilidad y simplicidad del sistema.

## 7. Metodología de trabajo

Para el desarrollo del proyecto se adoptó un marco de trabajo ágil basado en Scrum, estructurado en tres *Sprints*. Esta metodología permitió una entrega óptima, bien estructurada y adaptabilidad ante cambios en los requerimientos y una validación continua de las funcionalidades.

## 7.1. Estructura del proceso

- **Planificación de Sprints:** Al inicio de cada ciclo se definieron las historias de usuario y funciones prioritarias, asignando tareas específicas a cada integrante.
- **Desarrollo incremental:** Implementación modular del código fuente localizado en la carpeta `src/`, abarcando desde la ingesta de datos hasta la visualización.
- **Aseguramiento de calidad (QA):** Construcción en paralelo con pruebas unitarias mediante la librería `pytest` para validar el correcto filtrado y procesamiento de los datos, para posteriormente consolidarlas con las pruebas de integración para todo el sistema.

## 7.2. Herramientas y entorno de desarrollo

- **Control de versiones y colaboración:** Uso de **Git** y **GitHub** como repositorio central, aplicando ramas de trabajo y revisiones de código.
- **Asistencia con Inteligencia Artificial:** Utilización de asistentes de IA para soporte en la optimización de algoritmos, redacción y documentación.

## 7.3. Burndown Chart

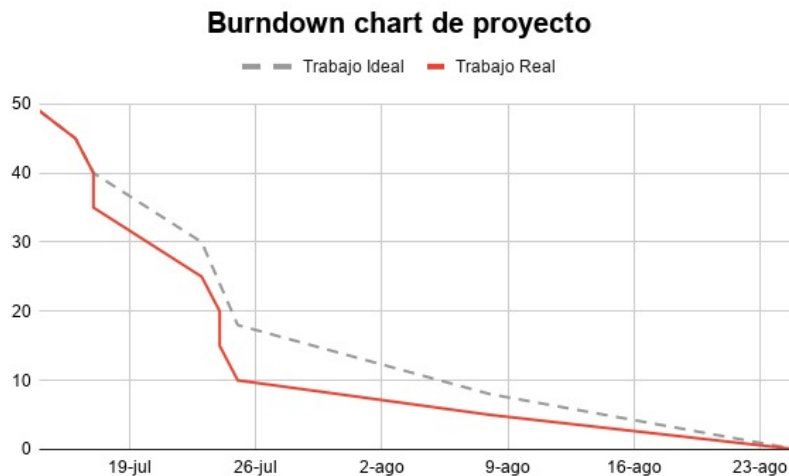


Figura 1: Gráfico Burndown Chart

**Aclaración sobre el trabajo previo y el Sprint Backlog:** Es importante señalar que, previo al inicio formal del Sprint 1, el equipo llevó a cabo una fase preliminar de exploración técnica y prototipado. Este trabajo previo no fue contabilizado dentro de las métricas de esfuerzo ni en la capacidad de los Sprints académicos. En consecuencia, los gráficos *Burndown Chart* presentados representan de manera exacta e independiente únicamente el trabajo planificado, asignado y completado dentro de cada iteración oficial del proyecto.



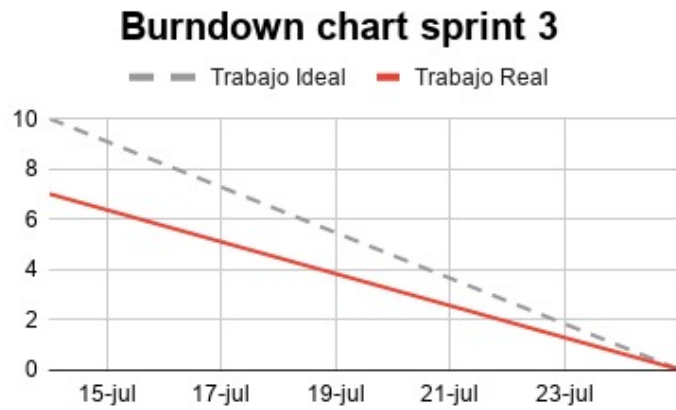


Figura 2: Elaboración Propia mediante Excel



Figura 3: Elaboración Propia mediante Excel

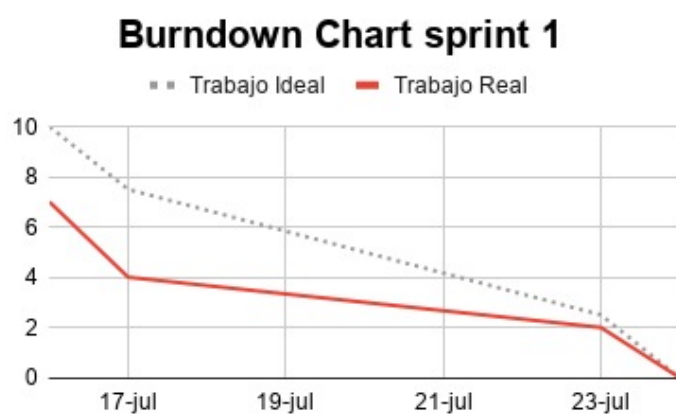


Figura 4: Elaboración Propia mediante Excel

## 7.4. Gestión del Backlog y Cierre de Historias de Usuario

### 7.4.1. Product Backlog (Estado Final)

A lo largo de los tres Sprints, el *Product Backlog* evolucionó para adaptarse a las necesidades técnicas y de la cliente. Al cierre del Sprint 3, el estado de las Historias de Usuario (HU) es el siguiente:

- **HU1 - Exportación de datos (.txt):** Completada (Sprint 3).
- **HU2 - Filtrado temporal:** Completada (Sprint 1 y 3).
- **HU3 - Trazabilidad de la fuente:** Completada (Sprint 3).
- **HU4 - Conversión de unidades ( $R_S$ ,  $AU$ ,  $R_E$ ):** Completada (Sprint 3).
- **HU5 a HU6 - Estructuración, limpieza y descarga automatizada:** Completadas (Sprint 2).
- **HU7 a HU8 - Interfaz web e interacción:** Completadas (Sprint 3).
- **HU9 a HU11 - Arquitectura, Pytest y CI/CD:** Completadas (Sprint 2 y 3).

### 7.4.2. Sprint Backlog (Sprint 3)

Para esta última iteración, el equipo se enfocó en acoplar los desarrollos previos y finalizar la capa de presentación. Las tareas críticas abordadas y cerradas en este ciclo fueron:

1. **TSK-07 — Interfaz web e integración frontend:** Desarrollo de la lógica en JavaScript para la lectura del JSON local, conversión de unidades matemáticas en tiempo real y renderizado de la tabla interactiva. (*Completado*)
2. **TSK-08 — Módulo de exportación:** Implementación del controlador que permite descargar los datos filtrados en formato `.txt` utilizando objetos *Web Blob*. (*Completado*)
3. **TSK-09 — Pruebas finales y documentación:** Refactorización del código para asegurar el tiempo de respuesta (RNF4), validación end-to-end de la integración Python-HTML y actualización del README.md final. (*Completado*)

## 8. Diseño de la Arquitectura

Para la entrega final del Sprint 3, la arquitectura del sistema ha pasado de una etapa conceptual a una implementación funcional y desplegada. El sistema se basa en un modelo monolítico ligero centrado en el procesamiento eficiente de datos espaciales.

### Componentes Principales del Sistema:

- **Capa de Presentación (Frontend):** Desarrollada en HTML5/CSS3 nativo (`index.html`). Permite a la investigadora seleccionar rangos de fechas, elegir la unidad de conversión de distancia y visualizar los datos filtrados en una tabla interactiva previa a la descarga.

- **Capa de Lógica y Procesamiento (Backend):** Desarrollada completamente en Python. Contiene los módulos principales:
  - `descargar_data.py`: Módulo encargado de gestionar las peticiones HTTP/API hacia la fuente oficial de la NASA mediante las librerías `requests` y `cdflib`.
  - `Filtrar_data.py`: Implementa la lógica de negocio utilizando la librería `Pandas` para la limpieza, manipulación de bloques de memoria y conversión matemática de unidades (radios solares, astronómicas y terrestres).
  - `visualizar_data.py`: Genera la estructura de datos compatible con la interfaz web y prepara la exportación final.
- **Capa de Datos Externa:** Conexión directa a los repositorios públicos de la NASA (CDAWeb / SPDF) para la extracción de archivos en formato crudo CDF (*Common Data Format*).
- **Módulo de Exportación:** Generador de archivos de texto plano (`.txt`) formateados para su integración directa en scripts de análisis de datos de terceros.

## 8.1. Atributos de Calidad y Rendimiento Realizados

- **Tiempo de Respuesta (RFN 4):** Se aplicó filtrado vectorial en `Pandas` y segmentación en bloques durante la lectura de archivos CDF, logrando procesar y renderizar las consultas dentro del límite estipulado de 7 segundos.
- **Estructura del Repositorio:** Se consolidó la estructura modular del código fuente dentro de la carpeta `src/`, aislando la interfaz web y documentando las dependencias necesarias en el archivo `README.md`.

## 8.2. Código fuente completo y ejecutable en GitHub

El código fuente completo de nuestro proyecto se encuentra disponible públicamente en el siguiente repositorio de GitHub:

**Repositorio:** <https://github.com/thiaremolina/Proyecto-Sonda-Parker-Desoft>

El repositorio incluye:

```
Proyecto-Sonda-Parker-Desoft/
.github/
  workflows/
docs/                                # Documentación
  Informe Sprint 2.pdf
  Sprint 1.pdf
src/                                  # Código fuente principal en Python
  Filtrar_data.py                    # Lógica de filtrado y procesamiento
  descargar_data.py                  # Script de conexión y descarga de la NASA
  visualizar_data.py                 # Script para la generación de gráficos/datos
  index.html                         # Interfaz gráfica web principal
  psp_datos_filtrados.json
  psp_visualizacion.png
  visualizar_data.py
```

```

test/                                # Pruebas hechas con pytest y pipeline
  test_descargar_data.py
  test_filtrar_data.py
.gitignore                           # Archivos excluidos del control de versiones
README.md                           # Documentación e instrucciones de uso
pyproject.toml
requirements.txt

```

## 9. Flujo de Ejecución (Implementación)

La implementación de la arquitectura descrita anteriormente opera bajo un flujo de tres etapas secuenciales:

1. **Ingesta de datos:** El cliente HTTP se conecta a la NASA, valida el espacio en disco local y descarga exclusivamente los archivos `.cdf` faltantes, optimizando el ancho de banda y el tiempo de espera.
2. **Procesamiento vectorial:** El sistema lee los datos astrofísicos y utiliza `Pandas` para filtrar valores nulos. Aquí ocurre el cálculo crítico: se remuestrea la serie temporal a intervalos de 1 hora mediante interpolación lineal (para cubrir los vacíos de transmisión de la sonda) y se exporta un archivo `JSON` intermedio.
3. **Presentación y Consumo:** La interfaz ligera lee el `JSON`, aplica las conversiones matemáticas en tiempo real ( $R_S$ ,  $AU$ ,  $R_E$ ) según lo solicite el usuario y habilita la descarga tabular plana, dejando los datos listos para el análisis científico.

## 10. Pruebas unitarias y/o de integración

Con el objetivo de garantizar la calidad y estabilidad del código antes de su integración a la rama principal, se implementaron 10 pruebas unitarias más, las cuales tratan de:

- `descargar_data.py` (7 pruebas): Se agregaron 2 más a comparación del Sprint 2.
- `filtrar_data.py` (13 pruebas): Se agregaron 8 más a comparación del Sprint 2.

Además, se siguen integrando con pipeline de integración Continua mediante Github Actions, que ejecuta automáticamente las 20 pruebas ante cada push o pull request dirigido a las ramas `main` y `develop`, evitando que un cambio nuevo rompa una funcionalidad ya validada.

### 10.1. Pipeline de integración continua

Configurado en `.github/workflows/main.yml`.

El pipeline se ejecuta automáticamente cada vez que se realiza un push o pull request al repositorio, y realiza los siguientes pasos:

- Instala Python y las dependencias del proyecto.

- Ejecuta las pruebas automatizadas sobre las funciones de `descargar_data.py` y `Filtrar_data.py`.
- Da aviso del resultado, si fue exitoso o fallido en GitHub.  
A continuación se muestra una captura del pipeline ejecutándose correctamente:

```
PS C:\Users\barba\OneDrive\Desktop\Proyecto-Sonda-Parker-Desoft> pytest -v pruebas
===== test session starts =====
platform win32 -- Python 3.13.2, pytest-9.1.1, pluggy-1.6.0 -- C:\Users\barba\AppData\Local\Programs\Python\Python313\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\barba\OneDrive\Desktop\Proyecto-Sonda-Parker-Desoft
configfile: pyproject.toml
plugins: anyio-4.9.0
collected 20 items

pruebas/test_descargar_data.py::test_listar_archivos_del_año_ok PASSED [ 5%]
pruebas/test_descargar_data.py::test_listar_archivos_del_año_error_de_red PASSED [ 10%]
pruebas/test_descargar_data.py::test_descargar_salta_si_ya_existe PASSED [ 15%]
pruebas/test_descargar_data.py::test_descargar_nuevo_archivo PASSED [ 20%]
pruebas/test_descargar_data.py::test_descargar_borra_archivo_incompleto_si_falla PASSED [ 25%]
pruebas/test_descargar_data.py::test_espacio_libre_gb PASSED [ 30%]
pruebas/test_descargar_data.py::test_descargar_posicion_ya_existe PASSED [ 35%]
pruebas/test_filtrar_data.py::test_limpiar_datos_saca_valores_invalidos PASSED [ 40%]
pruebas/test_filtrar_data.py::test_limpiar_datos_df_vacio PASSED [ 45%]
pruebas/test_filtrar_data.py::test_detectar_perihelios_encuentra_minimo_local PASSED [ 50%]
pruebas/test_filtrar_data.py::test_detectar_perihelios_sin_minimos_claros PASSED [ 55%]
pruebas/test_filtrar_data.py::test_asignar_orbitas_numeras_correctamente PASSED [ 60%]
pruebas/test_filtrar_data.py::test_asignar_orbitas_sin_perihelios PASSED [ 65%]
pruebas/test_filtrar_data.py::test_preparar_datos_para_web_resampllea_y_rellena PASSED [ 70%]
pruebas/test_filtrar_data.py::test_listar_archivos_cdf_excluye_archivo_de_posicion PASSED [ 75%]
pruebas/test_filtrar_data.py::test_cargar_archivos_cdf_con_todas_las_variables PASSED [ 80%]
pruebas/test_filtrar_data.py::test_cargar_archivos_cdf_variable_faltante_usa_nan PASSED [ 85%]
pruebas/test_filtrar_data.py::test_cargar_archivos_cdf_error_en_un_archivo_no_rompe_todo PASSED [ 90%]
pruebas/test_filtrar_data.py::test_agregar_distancia_mockando_archivo_posicion PASSED [ 95%]
pruebas/test_filtrar_data.py::test_agregar_distancia_si_falla_agrega_nan PASSED [100%]

===== 20 passed in 4.85s =====
```

Figura 5: Funcionamiento correcto de test

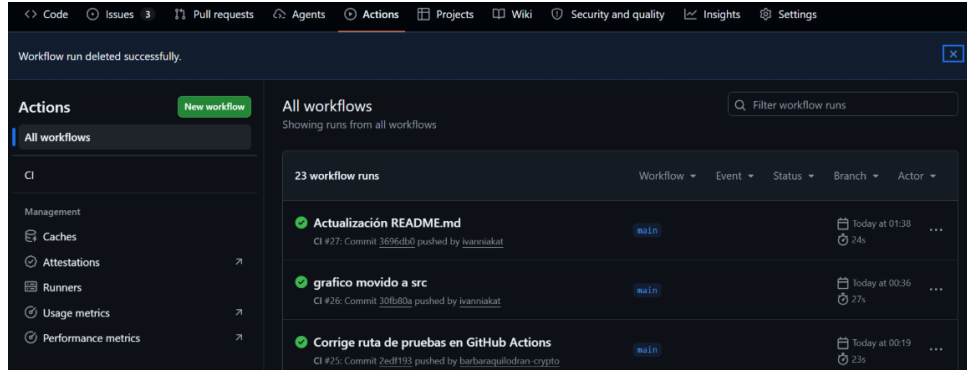


Figura 6: Correcto Funcionamiento

## 11. Pruebas pytest

Explicación de las 20 pruebas unitarias:

- `test_listar_archivos_del_año_ok`: Verifica que se identifiquen correctamente los archivos `.cdf` disponibles para un año determinado.
- `test_listar_archivos_del_año_error_de_red`: Comprueba el manejo de errores cuando ocurre un problema de conexión.
- `test_descargar_salta_si_ya_existe`: Verifica que un archivo existente no vuelva a descargarse.

- `test_descargar_nuevo_archivo`: Comprueba que un archivo nuevo sea descargado y almacenado correctamente.
- `test_descargar_borra_archivo_incompleto_si_falla`: Verifica que un archivo incompleto sea eliminado cuando ocurre un error durante la descarga.
- `test_espacio_libre_gb`: Comprueba que se obtenga correctamente el espacio disponible en disco.
- `test_descargar_posicion_ya_existe`: Verifica que el archivo de posición no se descargue nuevamente si ya existe.
- `test_limpiar_datos_saca_valores_invalidos`: Comprueba que se eliminen correctamente los valores inválidos de los datos.
- `test_limpiar_datos_df_vacio`: Verifica que la función procese correctamente un Data-Frame vacío.
- `test_detectar_perihelios_encuentra_minimo_local`: Comprueba que se detecte correctamente un mínimo local correspondiente a un perihelio.
- `test_detectar_perihelios_sin_minimos_claros`: Verifica que la función pueda procesar datos sin mínimos locales estrictos sin generar errores.
- `test_asignar_orbitas_numera_correctamente`: Comprueba que los datos sean asignados correctamente a las distintas órbitas.
- `test_asignar_orbitas_sin_perihelios`: Verifica el comportamiento cuando no existen fechas de perihelios.
- `test_preparar_datos_para_web_resampla_y_rellena`: Comprueba el remuestreo de datos y la interpolación de valores faltantes.
- `test_listar_archivos_cdf_excluye_archivo_de_posicion`: Verifica que se listen los archivos .cdf correspondientes a los datos, excluyendo el archivo de posición.
- `test_cargar_archivos_cdf_con_todas_las_variables`: Comprueba la carga correcta de archivos CDF con todas las variables requeridas.
- `test_cargar_archivos_cdf_variable_faltante_usa_nan`: Verifica que las variables ausentes sean reemplazadas por NaN.
- `test_cargar_archivos_cdf_error_en_un_archivo_no_rompe_todo`: Comprueba que un archivo CDF con errores no interrumpa todo el procesamiento.
- `test_agregar_distancia_mockando_archivo_posicion`: Verifica el cálculo y agregado de la distancia de la sonda al Sol.
- `test_agregar_distancia_si_falla_agrega_nan`: Comprueba que, ante un error con el archivo de posición, se agregue NaN sin detener el procesamiento.

## 12. Archivo `pyproject.toml`

Con el fin de garantizar la reproducibilidad del entorno de desarrollo y la fácil instalación del paquete, se configuró el archivo `pyproject.toml`.

Este documento define los metadatos del software, exige una versión mínima de Python (3.10) y gestiona tanto las dependencias principales (`pandas`, `requests`, `cdflib`, `Beautifulsoup4`, `Numpy`, `Os`, `Shutil`, `Json` ) como las herramientas de desarrollo (`pytest`). Esto facilita enormemente la instalación para cualquier investigador que clone el repositorio en el futuro.

## 13. Manual Básico de Uso

### 13.1. Requisitos Previos del Sistema

Para garantizar la correcta ejecución del software, el entorno debe contar con:

- **Python:** Versión 3.9 o superior.
- **Almacenamiento:** Mínimo 5 GB de espacio libre en disco para los archivos `.cdf`.
- **Navegador Web:** Google Chrome, Microsoft Edge o Mozilla Firefox.

### 13.2. Instalación de Dependencias

1. Clonar el repositorio desde GitHub e ingresar a la carpeta del proyecto.
2. Instalar las librerías necesarias ejecutando en la terminal:

```
pip install requests beautifulsoup4 cdflib pandas numpy
```

### 13.3. Flujo de Ejecución y Uso

1. **Descarga de datos crudos:** Ejecutar el script para obtener los archivos desde la NASA:

```
python src/descargar_data.py
```

2. **Procesamiento y limpieza:** Generar los archivos filtrados (`.csv` y `.json`):

```
python src/Filtrar_data.py
```

3. **Visualización en la Web:**

- Abrir el archivo `index.html` utilizando un servidor local (se recomienda la extensión *Live Server* en VS Code para evitar bloqueos CORS).
- Seleccionar el rango de fechas deseado mediante el filtro temporal.
- Seleccionar la unidad de distancia preferida (radios solares, UA o radios terrestres).

- Hacer clic en el botón de exportación para guardar la tabla en formato `.txt`.
- **Paso 1** Descarga de datos: Descarga los archivos `.cdf` de la sonda desde la NASA, evitando volver a descargar archivos ya existentes y avisando si no cuenta con espacio suficiente en su dispositivo.

Colocando en la terminal: `python src\descargar_data.py`

```
PS C:\Users\barba\OneDrive\Desktop\Proyecto-Sonda-Parker-Desoft> python src\descargar_data.py
=====
psp sweep -- descarga completa 2018-2025
=====
```

Figura 7: Terminal: `python src\descargar_data.py`

- **Paso 2** Procesamiento de datos *Filtrar Data*: Procesa los archivos `.cdf`, calcula distancias y órbitas, y genera `psp_datos_filtrados.csv` y `psp_datos_filtrados.json`

Colocando en la terminal: `python src\Filtrar_data.py`

```
PS C:\Users\barba\OneDrive\Desktop\Proyecto-Sonda-Parker-Desoft> python src\Filtrar_data.py
=====
INICIANDO PROCESAMIENTO DE DATOS PSP
=====

Carpeta src:
C:\Users\barba\OneDrive\Desktop\Proyecto-Sonda-Parker-Desoft\src
```

Figura 8: Procesamiento de datos(Filtrar Data)

- **Paso 3** Visualización en la web: Se abre `index.html` mediante un servidor local (Live Server), lo que carga automáticamente los datos y permite
  - Filtrar por rango de fechas
  - Cambiar la unidad de distancia
  - Descargar los datos filtrados en formato `.txt`.



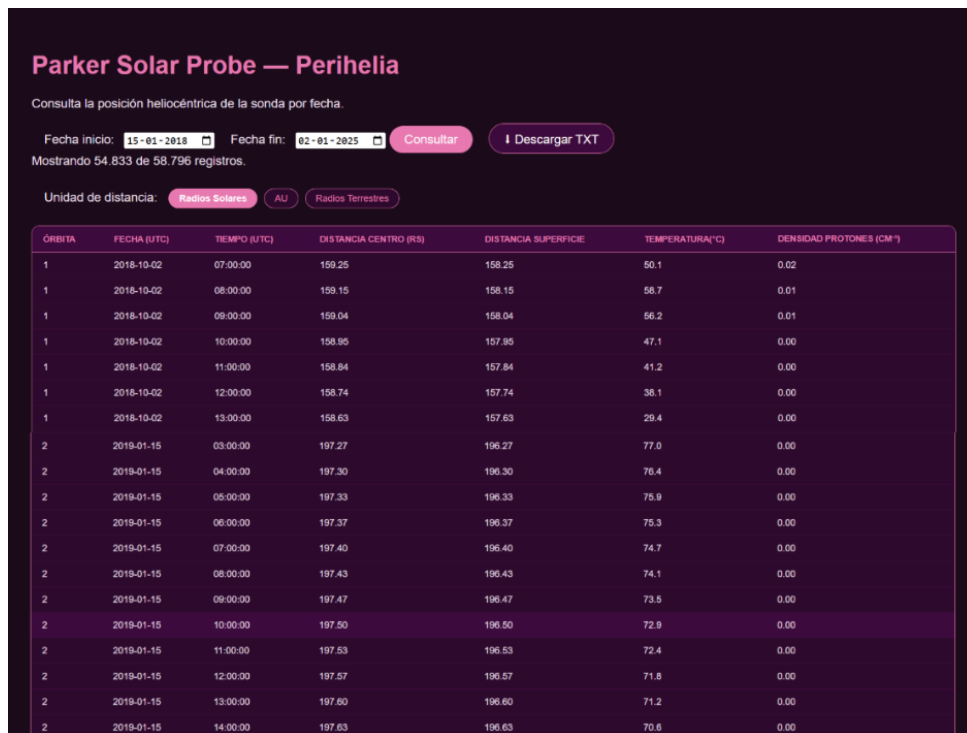


Figura 9: Captura de Visualización

| Orbita | Fecha (UTC) | Tiempo (UTC) | Dist. centro (Rs) | Dist. superficie (Rs) | Temp (C) | Densidad (cm-3) |
|--------|-------------|--------------|-------------------|-----------------------|----------|-----------------|
| 1      | 2018-10-02  | 07:00:00     | 159.25            | 158.25                | 50.1     | 0.02            |
| 1      | 2018-10-02  | 08:00:00     | 159.15            | 158.15                | 58.7     | 0.01            |
| 1      | 2018-10-02  | 09:00:00     | 159.04            | 158.04                | 56.2     | 0.01            |
| 1      | 2018-10-02  | 10:00:00     | 158.95            | 157.95                | 47.1     | 0.00            |
| 1      | 2018-10-02  | 11:00:00     | 158.84            | 157.84                | 41.2     | 0.00            |
| 1      | 2018-10-02  | 12:00:00     | 158.74            | 157.74                | 38.1     | 0.00            |
| 1      | 2018-10-02  | 13:00:00     | 158.63            | 157.63                | 29.4     | 0.00            |
| 1      | 2018-10-02  | 14:00:00     | 158.54            | 157.54                | 43.2     | 0.00            |
| 1      | 2018-10-02  | 15:00:00     | 158.44            | 157.44                | 30.4     | 0.01            |
| 1      | 2018-10-02  | 16:00:00     | 158.33            | 157.33                | 26.2     | 0.01            |
| 1      | 2018-10-02  | 17:00:00     | 158.22            | 157.22                | 23.4     | 0.01            |
| 1      | 2018-10-02  | 18:00:00     | 158.13            | 157.13                | 40.2     | 0.00            |
| 1      | 2018-10-02  | 19:00:00     | 158.03            | 157.03                | 21.7     | 0.00            |
| 1      | 2018-10-02  | 20:00:00     | 157.92            | 156.92                | 22.2     | 0.00            |
| 1      | 2018-10-02  | 21:00:00     | 157.82            | 156.82                | 22.6     | 0.00            |
| 1      | 2018-10-02  | 22:00:00     | 157.72            | 156.72                | 23.1     | 0.00            |
| 1      | 2018-10-02  | 23:00:00     | 157.61            | 156.61                | 23.6     | 0.00            |
| 1      | 2018-10-03  | 00:00:00     | 157.51            | 156.51                | 24.0     | 0.00            |
| 1      | 2018-10-03  | 01:00:00     | 157.41            | 156.41                | 24.5     | 0.00            |
| 1      | 2018-10-03  | 02:00:00     | 157.30            | 156.30                | 25.0     | 0.00            |
| 1      | 2018-10-03  | 03:00:00     | 157.20            | 156.20                | 25.4     | 0.00            |
| 1      | 2018-10-03  | 04:00:00     | 157.10            | 156.10                | 25.9     | 0.00            |
| 1      | 2018-10-03  | 05:00:00     | 156.99            | 155.99                | 26.4     | 0.00            |
| 1      | 2018-10-03  | 06:00:00     | 156.89            | 155.89                | 36.3     | 0.00            |
| 1      | 2018-10-03  | 07:00:00     | 156.78            | 155.78                | 22.4     | 0.00            |
| 1      | 2018-10-03  | 08:00:00     | 156.67            | 155.67                | 26.8     | 0.00            |
| 1      | 2018-10-03  | 09:00:00     | 156.56            | 155.56                | 58.7     | 0.01            |
| 1      | 2018-10-03  | 10:00:00     | 156.46            | 155.46                | 49.8     | 0.01            |
| 1      | 2018-10-03  | 11:00:00     | 156.35            | 155.35                | 80.7     | 2.06            |
| 1      | 2018-10-03  | 12:00:00     | 156.25            | 155.25                | 72.8     | 1.80            |
| 1      | 2018-10-03  | 13:00:00     | 156.15            | 155.15                | 64.8     | 1.54            |

Figura 10: Archivo .txt Descargado

## 14. Limitaciones de la solución

El sistema desarrollado para el análisis y visualización de datos de la Sonda Solar Parker presenta las siguientes limitaciones:

- Rigidez en los Formatos de Entrada: Las funciones de lectura de datos (descargar\_data.py y Filtrar\_data.py) dependen estrictamente de esquemas de datos prede-

finidos. Variaciones no documentadas en los encabezados o en el formato de origen requieren modificaciones manuales en la lógica del código.

- **Dependencia de recursos del cliente local:** El procesamiento de grandes volúmenes de datos y la generación de gráficos (`visualizar_data.py`) se ejecutan de manera local, por lo que la velocidad de ejecución está acotada a las especificaciones de hardware (CPU y memoria RAM) de la máquina que ejecuta el script.
- **Compatibilidad de Sistema Operativo (Enfoque en el Valor del Cliente):** Durante las retroalimentaciones previas, se sugirió probar la herramienta en entornos macOS y Linux. Sin embargo, aplicando los principios ágiles de priorizar el valor directo para el usuario, y dado que la investigadora (Product Owner) opera exclusivamente bajo el entorno Windows, el equipo decidió concentrar los esfuerzos del Sprint 3 en finalizar las características críticas (interfaz y exportación de datos). En consecuencia, el funcionamiento y estabilidad de la aplicación están garantizados y probados formalmente solo para Windows.

## 15. Conclusiones del proyecto

Para finalizar el Sprint 3, se ha cumplido exitosamente con el desarrollo, validación y entrega del software. El software entregado responde de manera eficaz y estable a los requerimientos definidos con el cliente, ofreciendo una herramienta funcional para el análisis de datos espaciales y dejando una base sólida para futuras expansiones del proyecto.

## 16. Anexos

### 16.1. Declaración de uso de Inteligencia Artificial

Como parte de nuestro flujo de trabajo moderno, utilizamos herramientas de Inteligencia Artificial de manera responsable y transparente para potenciar el desarrollo del proyecto.

- **Herramientas utilizadas:** Modelos de lenguaje (ChatGPT, Claude, Gemini) y asistentes de autocompletado de código (GitHub Copilot).
- **Casos de uso técnico:** Empleamos estas herramientas principalmente para generar la estructura *boilerplate* de las pruebas unitarias en `pytest` (mocks) y como apoyo durante la refactorización para optimizar el filtrado vectorial en Pandas.
- **Apoyo en redacción:** Se utilizó asistencia de IA para mejorar la cohesión, ortografía y estilo de redacción tanto en este informe final como en la documentación del `README.md`.

**Responsabilidad del equipo:** Consideramos la IA como una herramienta de apoyo, no como un reemplazo del criterio de ingeniería. Todo el código, configuraciones y textos sugeridos por estas herramientas fueron analizados, probados e integrados manualmente por las integrantes del equipo, asumiendo la autoría y responsabilidad total sobre el funcionamiento del software entregado.

## Referencias

- [1] National Aeronautics and Space Administration [NASA]. (s.f.). *Coordinated Data Analysis Web (CDAWeb) - Space Physics Data Facility (SPDF)*. <https://cdaweb.gsfc.nasa.gov/>
- [2] Google. (2026). *Gemini* (Versión 2026) [Modelo de lenguaje grande]. <https://gemini.google.com/>
- [3] National Aeronautics and Space Administration [NASA]. (s.f.). *Parker Solar Probe*. NASA Science. <https://science.nasa.gov/mission/parker-solar-probe/>
- [4] Universidad de Santiago de Chile [USACH]. (s.f.). *Matilde Coello*. Departamento de Física. <https://fisica.usach.cl/es/matilde-coello>
- [5] National Aeronautics and Space Administration [NASA]. (s.f.). *Space Physics Data Facility (SPDF)*. <https://spdf.gsfc.nasa.gov/pub/data/>
- [6] Johns Hopkins Applied Physics Laboratory. (s.f.). *The mission - Parker Solar Probe: Spacecraft and Instruments*. <https://parkersolarprobe.jhuapl.edu/>
- [7] Maffeo, M., & Harter, B. (2024). *cdflib: A Python module to read/write Common Data Format (CDF) files*. Python Package Index (PyPI). <https://pypi.org/project/cdflib/>
- [8] Krekel, H., Oliveira, B., Pfannschmidt, F., Bruynooghe, F., Laughner, T., & Schlammer, F. (2024). *pytest: helps you write better programs* (Versión 8.x) [Software framework]. <https://docs.pytest.org/>
- [9] GitHub. (2024). *GitHub Actions Documentation: Automating your workflow with CI/CD*. GitHub Docs. <https://docs.github.com/en/actions>
- [10] McKinney, W., & The pandas development team. (2024). *pandas-dev/pandas: pandas* (Versión 2.x) [Software de análisis de datos]. Zenodo. <https://pandas.pydata.org/>
- [11] Richardson, L. (2024). *Beautiful Soup Documentation* (Versión 4.x). Crummy. <https://www.crummy.com/software/BeautifulSoup/bs4/doc/>